

CSC322: A Modern Programming Language

Design and Development of a Calculation Engine

Deadline

Final Submission: on or before 11:59pm on Friday, August 7, 2026

Problem definition

Spreadsheets are part of our everyday life and a component of almost all administrative processes, especially the computation of students' results. Every semester, the results of thousands of students pass through several Excel workbooks, one per course, per department and per level. Grades are computed with formulas that nobody remembers writing, copied from sheet to sheet, until a single wrong cell reference quietly corrupts a student's CGPA. The University has approached the Department of Computer Science to help them out with a calculation engine that could be integrated into the University's results processing portal and other administrative systems. The department is throwing the challenge to its students because it believes in the creativity of its students. The department is willing to reward a unique, efficient and excellent calculation engine that could be easily integrated into the University's portal. CSC322 has exposed you to the design and implementation of abstract data type. You have been taught to always see a software design project as an opportunity to create user-defined type. Your solution should thus be in the form of an API which could be deployed and used by any client interested in implementing spreadsheet-style calculations. Think of it as the part of Excel you never see: no ribbon, no charts, just the machinery that makes `=SUM(B2:B45)*0.3` spring to life the moment a mark is entered.

Design specifications

- A formula language supporting numbers, text, cell references (e.g. *B2*), ranges (e.g. *B2:B45*), arithmetic and comparison operators, parentheses, and at least the following functions: SUM, AVERAGE, MIN, MAX, COUNT, IF, ROUND and LOOKUP. The grammar must be formally specified and implemented using a parser generator (the ANTLR C# target is recommended).
- Reactive recalculation: when one cell changes, only the cells that depend on it, directly or indirectly, are recomputed, and in a correct order.
- A single cell edit in a workbook of 100,000 cells with a chain of 500 dependent cells must propagate within 50 milliseconds. A full recalculation of the same workbook must complete within 2 seconds.
- Circular references must be detected and reported to the client with the exact cycle (e.g. *A1 to B3 to C7 back to A1*), never with a crash or an infinite loop.
- Undo and redo of at least the last 100 editing operations.
- Two additional features (e.g. date arithmetic, statistical functions, conditional formatting rules, CSV import) will be assigned to you by the course lecturer during the first week of the project. Assignments differ, so no two solutions will be exactly the same.

Purpose

The main aim of this project is to enable you to reinforce your knowledge and gain experience in:

- designing and implementing Abstract Data Type (ADT); the expression tree and the dependency graph are the heart of this project;

- using common design patterns; expect Composite and Interpreter in your expression tree, Observer in change propagation, Command in undo/redo, and Strategy or Factory in your function library;
- using a parser generator;
- directing, critiquing and taking full responsibility for code written with AI assistants;
- defending your design decisions, on paper, in code and in person.

Other objectives of this project include:

- illustrating the benefits of software engineering techniques;
- strengthening your ability to define, clarify and decompose problems;
- encouraging collaboration and team work.

Design process

A calculation engine is the heart of every spreadsheet. It is a small pipeline with a graph at its centre. When a client assigns a formula to a cell, the formula text is handed to a parser, which either rejects it with a helpful error message or produces an expression tree, a composite structure of literals, references, operators and function calls. From that tree the engine extracts the cell's dependencies and records them in a dependency graph. When the value of any cell changes, the engine consults the graph, determines every affected cell, orders them so that no cell is computed before the cells it depends on, evaluates them and notifies the client of each change.

Our work will be based on this simple abstraction of a calculation engine as given below:

- **Formula representation:** you need to design the grammar of the formula language and use your parser generator to build the expression tree. The tree is an ADT; state its abstraction function and representation invariant clearly. Remember that malformed input is normal input. Your error messages should tell the user what is wrong and where it is wrong.
- **Dependency tracking:** the dependency graph is the most important data structure in this project. The response time of your engine depends largely on the data structure used to represent this graph. It must support fast insertion and removal of edges as formulas change, detection of cycles, and topological ordering of affected cells.
- **Evaluation:** this is one of the most important aspects of the design. The evaluator walks the expression tree to compute values. It must handle type errors (text where a number is expected), missing references and division by zero the way a spreadsheet does, with error values, not with exceptions that escape to the client.
- **Results:** even though the major requirement in this project is to come up with a calculation engine API, you will however need to demonstrate that your API is efficient by creating a graphical user interface that allows the user to interact with the API. The interface should contain at least a scrollable grid in which a user can select a cell, type a value or a formula, and watch dependent cells update instantly. The grid should visibly flag error values and circular references.

Design approach

The general architecture of a calculation engine has been given above. For each of the modules, you are expected to do a proper design using class diagrams to show the components of the module and the relationship between them. This is very important and it should be done before you start coding. You are free to discuss your design with your course lecturer before you start the implementation. Remember to always state clearly the specification for your classes and methods. Test first programming style should be

adopted in the implementation of your design. Your git history should show tests being committed before or alongside the code that makes them pass, and it will be read.

Working with AI

You may use AI coding assistants (ChatGPT, Claude, Copilot and the like) anywhere in this project. Professional engineers do the same. But an engineer who ships code is responsible for that code, no matter who or what wrote the first draft. Three rules follow:

- **Understand everything you submit.** At the oral defence, you may be asked to explain any line of your codebase and to modify any part of it on the spot. Saying that the AI wrote that part is not an answer; it is a confession.
- **Log significant AI use.** Keep an AI Collaboration Log. For each significant use, record the date, the tool, what you asked for, what you got, what you changed and why. A log entry that says the dependency graph suggested by the AI was rejected because it recomputed the whole workbook on every edit will earn marks. A log that says it worked so we pasted it will lose marks. Undisclosed wholesale submission of AI-generated code is academic misconduct.
- **Complete the critique exercise.** Ask an AI assistant to produce a complete solution to one module of this project. Then write a two-page review of that solution the way a senior engineer would: where is it correct, where is it subtly wrong, where does its design fall short of the specifications above, and what did you do differently? Attach the transcript of your conversation with the AI.

Why these rules? Because the competencies this course certifies, which are decomposing a problem, choosing a representation, specifying an interface and judging code, are exactly the ones you need to use AI well rather than be used by it. The assessment below is designed so that anyone who outsources their thinking will be found out at the oral defence and by the assigned features that generic AI output cannot be made to fit.

Assessment

The project carries 100 marks, distributed as shown below:

Component	Weight	What we look for
Design portfolio	20	Class diagrams, ADT specifications (abstraction functions, representation invariants) and the formal grammar
Working system and benchmarks	25	Correctness against our hidden test suite; performance targets met and demonstrated; assigned features delivered
Tests and process evidence	15	Coverage and quality of your test suite; a git history showing test first work and steady progress
AI Collaboration Log and critique	10	Honesty and depth of the log; insight shown in the critique exercise
Oral defence and live modification	20	You explain your code on request and implement a small unseen change to your own engine, live
Reflection	10	Two pages: what you designed, what you would do differently, and what the AI tools got wrong

Deliverables

- Source code repository (C# class library and GUI client) with full git history;
- Design portfolio: class diagrams, specifications and the formal grammar;
- Test suite and instructions for running the performance benchmarks;
- AI Collaboration Log and the critique exercise with the transcript attached;
- A written reflection;
- A 5-minute demonstration video of the GUI client driving the API.

Background reading

You might need to read the following documents to familiarize yourself with spreadsheet internals, dependency graphs and parsing techniques:

- How to recalculate a spreadsheet
- <https://lord.io/spreadsheets/>
- Topological sorting
- https://en.wikipedia.org/wiki/Topological_sorting
- The ANTLR Mega Tutorial (C# target)
- <https://tomassetti.me/antlr-mega-tutorial/>
- Interpreter, Composite and Observer patterns
- https://en.wikipedia.org/wiki/Interpreter_pattern
- <https://refactoring.guru/design-patterns/composite>
- <https://refactoring.guru/design-patterns/observer>